

2026 Spring

Course ID: 1142-1765

Advanced Programming



05-Python programming for 'Binary Search'



Advanced Programming

Python · C++ · Rust

Data Structures & Algorithms



Python



C++



Rust

Part-1

Python
Algorithm

LeetCode Problem
“Binary Search”

Course Activities
Intro to CPE Exam



Python & C++ Algorithm

Linear Sort · Binary Sort

Comprehensive Guide to Data Structures Stack and Queue

Sorting Algorithms

Linear Sort · Binary Sort

Concepts · Algorithms · Complexity · Comparisons

Linear Sort

$O(n)$ time

Counting · Radix · Bucket

Binary Sort

$O(n \log n)$ time

Binary Insertion Sort

Agenda

01

What is Linear Sort?

Non-comparison-based sorting with $O(n)$ complexity

02

Counting Sort

Integer-key sorting by frequency counting

03

Radix Sort

Digit-by-digit stable sort

04

Bucket Sort

Distribute values into buckets and sort each

05

What is Binary Sort?

Binary Insertion Sort — fewer comparisons

06

Complexity Comparison

Time & space trade-offs at a glance

What is Linear Sort?

Key Insight: Linear sorts bypass the $O(n \log n)$ comparison-based lower bound by exploiting extra knowledge about the input data — such as key ranges, digit structure, or value distribution.

Non-Comparative

Never compare elements to each other
— avoids the $\Omega(n \log n)$ lower bound

Integer / Range Keys

Works on bounded integers, strings of fixed alphabet, or uniformly distributed reals

Linear Time $O(n)$

Achieves $O(n + k)$ or $O(n)$ time when the key range $k = O(n)$

Main Algorithms:

Counting Sort

Radix Sort

Bucket Sort

Counting Sort

$O(n + k)$

1

Count Frequencies

Build array $C[0..k]$ counting occurrences of each key value.

2

Cumulative Sum

Transform C so that $C[i]$ holds the number of elements $\leq i$.

3

Place Elements

Traverse input right-to-left; place each element in its correct output position.

Pseudocode

```
function countingSort(A, k):  
    C = array of (k+1) zeros  
    for j in A: C[j]++  
    for i = 1 to k:  
        C[i] += C[i-1]  
    B = output array  
    for j in reverse(A):  
        B[C[A[j]]-1] = A[j]  
        C[A[j]]--  
    return B
```

Time

$O(n+k)$

Space

$O(n+k)$

Stable?

Yes ✓

In-place?

No

Radix Sort

$O(d \cdot (n + k))$

Sort numbers digit-by-digit, from Least Significant Digit (LSD) to Most Significant Digit (MSD), using a stable sort (usually Counting Sort) at each digit position.

Example: Sort [170, 45, 75, 90, 802, 24, 2, 66]

Round 1
(Units digit)

170

090

802

002

024

045

075

066

Round 2
(Tens digit)

802

002

024

045

066

170

075

090

Round 3
(Hundreds)

002

024

045

066

075

090

170

802

Time $O(d \cdot n)$

Space $O(n + k)$

Stable? Yes ✓

Use case Fixed-len
keys

Bucket Sort

Avg $O(n)$

Distribute elements into buckets covering sub-ranges of the input, sort each bucket independently (often with Insertion Sort), then concatenate. Best when input is uniformly distributed.

1

Create n Buckets

Divide $[0, 1)$ into n equal intervals. Each bucket covers a sub-range.

2

Distribute

For each element $a[i]$, place it in bucket $\lfloor n \cdot a[i] \rfloor$.

3

Sort Each Bucket

Sort each non-empty bucket (typically Insertion Sort).

4

Concatenate

Append all buckets in order to produce the sorted output.

⚠ Worst case $O(n^2)$ when all elements fall into one bucket | Average $O(n)$ with uniform distribution

Binary Sort (Binary Insertion Sort)

Binary Insertion Sort improves classical Insertion Sort by using Binary Search to find the correct insertion position — reducing comparisons from $O(n)$ to $O(\log n)$ per element, while keeping $O(n^2)$ shifts.

- 1 Start with sorted sub-array $A[0..i-1]$
- 2 Use Binary Search to find position pos for $A[i]$
- 3 Shift elements $A[pos..i-1]$ one position right
- 4 Insert $A[i]$ at pos ; advance i

Pseudocode

```
for i = 1 to n-1:
    key = A[i]
    pos = binarySearch(
        A, 0, i-1, key)
    j = i - 1
    while j >= pos:
        A[j+1] = A[j]
        j--
    A[pos] = key
```

Comparisons
 $O(n \log n)$

Shifts/Swaps
 $O(n^2)$

Space
 $O(1)$

Stable?
Yes ✓

Binary Search — The Core Subroutine

Given sorted sub-array $A[\text{lo}..\text{hi}]$, find insertion position for key using divide-and-conquer:

10	20	30	40	50	60	70
0	1	2	3	4	5	6

Search key = 37

1 $\text{lo}=0, \text{hi}=6 \rightarrow \text{mid}=3 \rightarrow A[\text{mid}]=40 \rightarrow 40 > 37 \rightarrow \text{search left}$

2 $\text{lo}=0, \text{hi}=2 \rightarrow \text{mid}=1 \rightarrow A[\text{mid}]=20 \rightarrow 20 < 37 \rightarrow \text{search right}$

3 $\text{lo}=2, \text{hi}=2 \rightarrow \text{mid}=2 \rightarrow A[\text{mid}]=30 \rightarrow 30 < 37 \rightarrow \text{insert at index 3}$

✓ Result: insert 37 at index 3 $\rightarrow [10, 20, 30, 37, 40, 50, 60, 70]$ | Only 3 comparisons vs 3 for linear scan (optimal for this input)

Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Stable?	In-Place?
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	✓	✗
Radix Sort	$O(d \cdot n)$	$O(d \cdot n)$	$O(d \cdot n)$	$O(n+k)$	✓	✗
Bucket Sort	$O(n+k)$	$O(n)$	$O(n^2)$	$O(n)$	✓	✗
Binary Insertion Sort	$O(n \log n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	✓
Standard Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	✓
Merge Sort (reference)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓	✗
Quick Sort (reference)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗	✓

 Linear Sort

 Binary Sort

 Reference

When to Use Which?

Use Counting Sort when...

- ▶ Key values are non-negative integers
- ▶ Range k is $O(n)$ or smaller
- ▶ Stable sort is required

Use Radix Sort when...

- ▶ Keys are integers or fixed-length strings
- ▶ Multiple passes over bounded digits
- ▶ Need guaranteed linear time

Use Bucket Sort when...

- ▶ Input is uniformly distributed reals in $[0,1)$
- ▶ Average case performance matters
- ▶ Memory is available for buckets

Use Binary Insertion Sort when...

- ▶ Array is small or nearly sorted
- ▶ Memory is very limited (in-place)
- ▶ Comparison count must be minimized

Rule of thumb: Linear sort when you know the key structure; Comparison sort when you don't.

Summary

Linear Sorts

- Break the $\Omega(n \log n)$ barrier
- Exploit key-range knowledge
- $O(n)$ time, $O(n+k)$ space
- Best for integers, fixed-range keys

Binary Sort

- Uses Binary Search for insertions
- $O(n \log n)$ comparisons
- Still $O(n^2)$ shifts — not faster overall
- Great for small or nearly-sorted arrays

Key Takeaways

- No single sort wins in all cases
- Know your data before choosing
- Stability & space matter in practice
- Hybrid approaches are common in industry



Introduction to

C++ Programming

with CodeChef

Learn programming fundamentals through competitive coding practice

```
#include<iostream>
using
namespace std;
int main() {
    cout <<
    "Hello, World!";
    return 0;
}
```


What is C++?

Fast & Powerful

Compiled to native machine code — one of the fastest languages.

System-Level Control

Direct memory management via pointers and references.

Object-Oriented

Supports classes, inheritance, polymorphism & encapsulation.

Industry Standard

Used in games, OS, browsers, embedded systems & competitive coding.

Why Learn C++ on CodeChef?

01

Free Online Compiler

No installation needed — write and run C++ directly in the browser IDE.

02

1000+ Practice Problems

Structured practice from beginner to advanced across all difficulty levels.

04

Learning Paths

C++ DSA courses, video tutorials, and editorial guides.

03

Monthly Contests

Participate in CodeChef Long, Cook-Off, and Lunchtime contests.

05

06

Getting Started on CodeChef

1

Create Account — Visit codechef.com and sign up with email or Google/GitHub.

2

Open IDE — Click "Practice" → select any problem → choose C++ (g++17) compiler.

3

Write Code — Type your C++ code in the editor with syntax highlighting support.

4

Compile & Run — Click "Run" to test with custom input before submitting.

5

Submit Solution — Click "Submit" — get Accepted (AC), WA, TLE or CE verdict instantly.

Your First C++ Program — "Hello, World!"



```
// Line comment: explains the code
#include<iostream>

using namespace std;

int main() {

    cout << "Hello, World!" << endl;
    return
    0;
}
```

#include <iostream>

Includes the I/O library that provides cout/cin.

using namespace std;

Lets us write cout instead of std::cout.

int main()

Entry point — every C++ program starts here.

"Hello, World!"

A string literal printed to the console.

return 0;

Signals the OS that the program ended successfully.

Variables & Data Types

Type	Size	Range / Example	Use Case
int	4 B	-2.1B to 2.1B e.g. int x = 42;	Integers, loop counters
long long	8 B	$\pm 9.2 \times 10^{18}$ e.g. 1LL << 60	Large numbers in contests
double	8 B	$\pm 1.7 \times 10^{308}$ e.g. 3.14159	Floating-point math
char	1 B	0–255 e.g. char c = 'A';	Single character
string	var	e.g. string s = "hello";	Text / word problems
bool	1 B	true / false	Flags and conditions



Tip: In competitive programming, prefer int for small numbers and long long when values exceed $\sim 10^9$

Input & Output in C++

cin — Reading Input



```
int n; // declare variable
cin >> n; // read one value

int a, b;
cin >> a >> b; // read two values

string s;
getline(cin, s); // full line
```

cout — Printing Output



```
cout << "Hello";
// → Hello

cout << a + b << endl;
// prints sum + newline

cout << fixed << setprecision(2)
    << 3.14159;
// → 3.14
```

Competitive Programming Tip — Fast I/O

```
ios_base::sync_with_stdio(false); cin.tie(NULL);
```

Control Flow — if / else / loops

if / else if / else

```
int x = 10;
if (x > 0) {
    cout << "Positive";
}
else if (x < 0) {
    cout << "Negative";
}
else {
    cout << "Zero";
}
```

for loop / while loop

```
// for loop: 1 to 5
for (int i=1; i<=5; i++) {
    cout << i << " ";
}
// Output: 1 2 3 4 5

// while loop
int n = 3;
while (n > 0) {
    cout << n-- <<
    " ";
}
```

switch — Use when comparing a variable against many fixed constants. **break;** is required to stop fall-through between cases.

Functions in C++

```

// Function declaration
int add(int a, int b) {
    return a + b;
}

// Void function (no return value)
void greet(string name) {
    cout << "Hi, " << name << endl;
}

int main() {
    greet(
        "Alice");    // → Hi, Alice
    cout << add(3, 4); // → 7
}

```



Return Type

Specifies what the function gives back. Use void if nothing is returned.



Parameters

Inputs passed to the function. Variables are local to the function scope.



Recursion

A function can call itself. Base case is essential to prevent infinite loops.



Overloading

Multiple functions with the same name but different parameter types.

Arrays & STL Essentials

C-style Array & std::vector

```
// Fixed array
int arr[5] = {1,2,3,4,5};
cout << arr[0]; // 1

// Dynamic vector
vector<int> v = {10, 20, 30};
v.push_back(40);
for (auto x : v)
    cout << x <<
    " ";
```

Essential STL Containers

vector<T>	Dynamic resizable array
stack<T>	LIFO stack (push/pop/top)
queue<T>	FIFO queue (push/pop/front)
map<K,V>	Sorted key-value pairs (O log n)
set<T>	Sorted unique elements (O log n)
priority_queue	Max-heap (great for Dijkstra)

#include <vector> | #include <map> | #include <set> | #include <queue> | #include <stack>

CodeChef Problem Walkthrough — "Sum of Two Numbers"

Problem Statement

Given two integers A and B, print their sum.

Input: Two integers A and B on one line.

Output: Print A + B on a new line.

Constraints:

$1 \leq A, B \leq 10^9$

Input

3 7

Output

10

```
#include<iostream>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    long long a, b;
    cin >> a >> b;
    cout << a + b << endl;
    return
0;
}
```

✓ ACCEPTED — AC

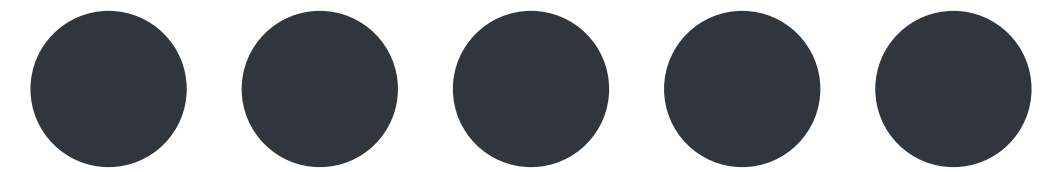
Your C++ Practice Roadmap on CodeChef

Phase 1 Basics	Phase 2 Data Structures	Phase 3 Algorithms	Phase 4 Advanced
Week 1–2 - I/O + Variables - If/Else + Loops - Functions - Beginner problems on CodeChef	Week 3–4 - Arrays & Strings - STL: vector, map, set - Recursion basics 1-star problems	Week 5–8 - Sorting & Searching - Binary Search - Two Pointers 2-star rated problems	Week 9+ - Dynamic Programming - Graph Algorithms - Segment Trees - Monthly Contests

 Consistency is key — solve at least 1 problem every day on CodeChef to build speed and confidence!

Ready to Code?

Start your C++ journey today on CodeChef



Platform

codechef.com



Learning

codechef.com/learn



Contests

Monthly competitions

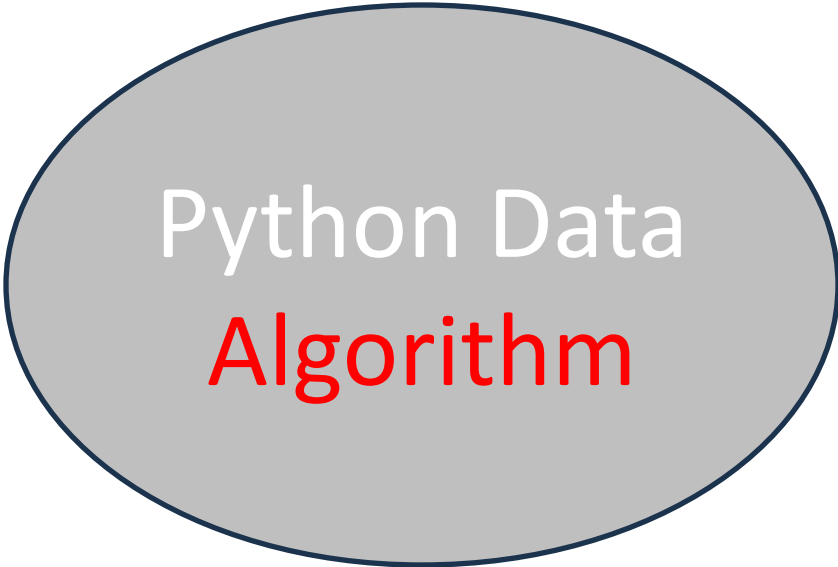


Community

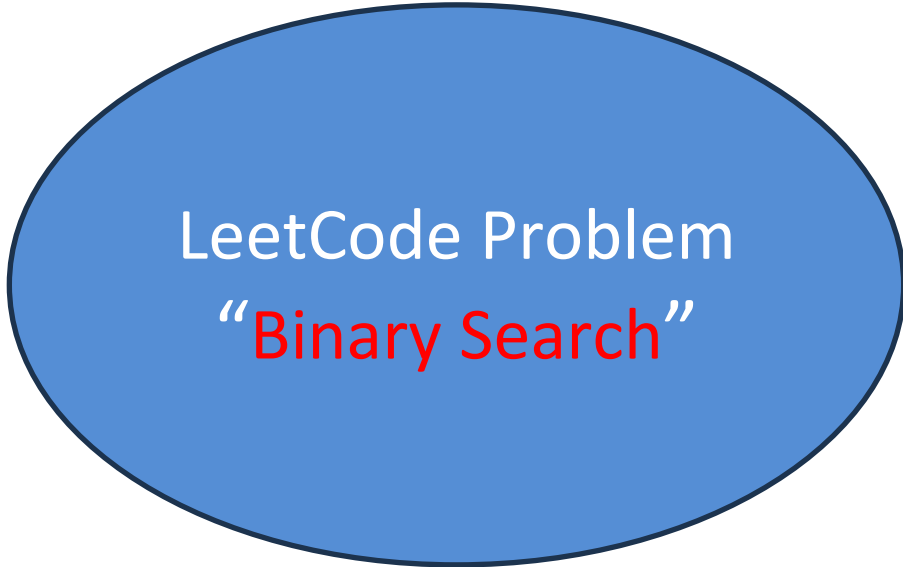
1M+ active coders

Happy Coding! 🎉 Keep practicing, keep improving

Part-2



Python Data
Algorithm



LeetCode Problem
“Binary Search”



Course Activities
CPE

LeetCode Practice

Master Programming Skills with Python · C++ · Rust

2500+

Problems

50+

Companies

10M+

Users

Difficulty Tiers



Easy



Medium



Hard

01. Two Sum

CO PRO PUAP01_Python_quick_tutorial.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

指令 + 程式碼 + 文字 | 全部執行 複製到雲端硬碟 連線

```
[ ] 1 instr = input('Please input your score: ')
    2 score = int(instr) #Integer
    3 print(score) # print the value of the variable
    4 print(type(score)) # print the type of the variable
```

Please input your score: 88
88
<class 'int'>

Arithmetic Operators and Expressions Arithmetic Operators

```
+ - * / #Add, subtract, multiply and divide
** #power
// #business
% #remainder
```

```
[ ] 1 print(1 + 2)
    2 print(3 - 4)
```

變數 終端機

<https://leetcode.com/problems/two-sum/description/>
<https://leetcode.cn/problems/two-sum/description/>

66. Plus One

The screenshot shows the LeetCode interface for the problem '66. Plus One'. The left pane contains the problem description, which states: 'You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the i^{th} digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's. Increment the large integer by one and return *the resulting array of digits*.' It also includes two examples: Example 1 with input `[1,2,3]` and output `[1,2,4]`, and Example 2 with input `[4,3,2,1]` and output `[4,3,2,2]`. The right pane shows a code editor with a Python3 solution:

```
1 class Solution:
2     def plusOne(self, digits: List[int]) -> List[int]:
3 
```

 Below the code editor, there is a 'Saved' status, a message 'You need to log in / sign up to run or submit', and tabs for 'Testcase' and 'Test Result'. At the bottom of the right pane, a message says 'You must run your code first'.

Problem List < > ↺

Submit

Description Editorial Solutions Submissions

66. Plus One

Easy Topics Companies

You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the i^{th} digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return *the resulting array of digits*.

Example 1:

Input: `digits = [1,2,3]`
Output: `[1,2,4]`
Explanation: The array represents the integer 123. Incrementing by one gives $123 + 1 = 124$. Thus, the result should be `[1,2,4]`.

Example 2:

Input: `digits = [4,3,2,1]`
Output: `[4,3,2,2]`

Code

Python3 • Auto

```
1 class Solution:
2     def plusOne(self, digits: List[int]) -> List[int]:
3 
```

Saved

You need to [log in / sign up](#) to run or submit

Testcase Test Result

You must run your code first

<https://leetcode.com/problems/plus-one/description/>

<https://leetcode.cn/problems/plus-one/description/>

20. Valid Parentheses

Description

Editorial

Solutions

Submissions

20. Valid Parentheses

Easy

Topics

Companies

Hint

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: `s = "()"`

Output: `true`

</> Code

Python3 • Auto

```
1 class Solution:
2     def isValid(self, s: str) -> bool:
3
```

Saved

You need to [log in / sign up](#) to run or submit

☒ Testcase

> Test Result

You must run your code first

<https://leetcode.com/problems/valid-parentheses/>
<https://leetcode.cn/problems/valid-parentheses/>

704. Binary Search

Problem List

<

>

↺

🔍

Submit

📄

🔗

👤

Register

or

👤

Log in

Premium

Description

Editorial

Solutions

Submissions

704. Binary Search

Easy

Topics

Companies

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12], target = 9`

Output: `4`

Explanation: 9 exists in `nums` and its index is 4

Example 2:

Input: `nums = [-1,0,3,5,9,12], target = 2`

Output: `-1`

Explanation: 2 does not exist in `nums` so return -1

Code

Python3 • Auto

1 class Solution:

2 def search(self, nums: List[int], target: int) -> int:

3

Saved

Ln 3, Col 9

You need to [log in / sign up](#) to run or submit

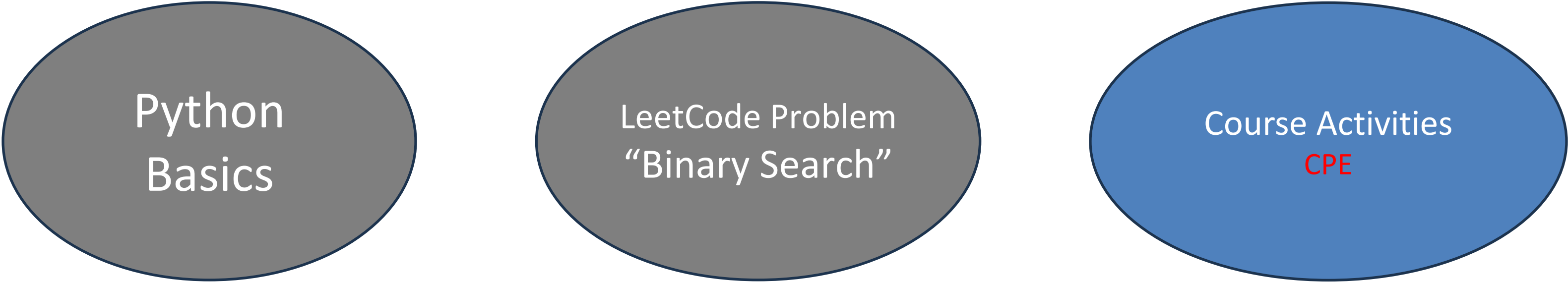
Testcase

Test Result

You must run your code first

<https://leetcode.com/problems/binary-search/>
<https://leetcode.cn/problems/binary-search/>

Part-3



Python
Basics

LeetCode Problem
"Binary Search"

Course Activities
CPE

OFFICIAL GUIDE

CPE Procedures

Collegiate Programming Examination · Proctor & Staff Reference

CPE verification dates

大學 程式能力檢定

Collegiate Programming Examination

英文

由「Google 翻譯」技術提供

[Introduction and Rules](#)

[Register or Login](#)

[Latest Exams](#)

[Environment and teaching materials](#)

[CPE Secrets](#)

[Past exams](#)

[Certificate of Academic Performance](#)

[Co-organizing Zone](#)

[Partner manufacturers](#)

[A complete introduction to the CPE](#)

[\(College Programming Ability Test\), key points for the application process, and examination rules.](#)

Following discussion and approval at the University Programming Proficiency Test (CPE) meeting on December 24, 2024, starting January 1, 2025, CPE certificate results will be sent entirely in electronic format.

Recent CPE verification dates are as follows:

Registration begins on December 9, 2025 (Tuesday) and will start on November 25, 2025 (Tuesday).

Registration begins on March 24, 2026 (Tuesday) and will start on March 10, 2026 (Tuesday).

Registration begins on May 26, 2026 (Tuesday) and will start on May 12, 2026 (Tuesday).

CPE can recommend 10 teams (starting in 2022) to the host university of ICPC Taiwan to participate in the ICPC competition.

[Registration website and deadline for recommended ICPC teams at CPE 2025](#)

1. Exam Schedule

17:00 – 17:20	Staff On-site	Login to JoinNet, obtain proctor password
17:30 – 18:00	Candidate Check-in	Verify ID, assign seats, distribute materials
17:40 – 18:30	Environment Test	Candidates test system; watch rules video at 18:25
18:00	Entry Deadline	No candidates admitted after this time
18:30 – 18:40	Break & Preparation	Final setup before formal exam
18:40 – 21:40	Formal Exam (3 hrs)	Official examination period begins
19:00	ID Verification	Proctors check candidate IDs & authentication codes
19:40+	Early Exit Allowed	Candidates may leave; must complete exit survey first

2. Pre-Exam Setup

① JoinNet Login

- Log in to JoinNet (<http://cu.nsysu.edu.tw/>) by 17:20
- Set up WebCam facing the exam room (for cross-venue monitoring)
- Use JoinNet for ALL communications during the exam

② Configure Proctor Machine

- Only ONE proctor machine per LAN segment
- Click 'Proctor Management' → enter proctor password
- Select the machine's IPv4 address → 'Set & Close'
- Verify all candidate machines show the login screen (press F5 if not)

③ Candidate Machines

- Power on candidate PCs and launch CodingFrenzy (candidate mode)
- Open only: [exam count] + 2 spare machines
- Do NOT activate spares unless one is actually used
- Set up a projector to display the scoreboard

3. Candidate Registration & Check-in

STEP 1 Verify Identity	Ask candidate to present: Student ID (university students) OR photo-bearing National ID / Health Insurance Card / Driver's License (non-university candidates)
STEP 2 Sign Attendance Sheet	Have each candidate sign the attendance sheet and be seated at their pre-assigned seat. Seating is NOT self-selected.
STEP 3 Distribute Materials	Give each candidate one copy of 'CPE Candidate Instructions' and one 'Exam Access Code' slip.
STEP 4 Record Incidents	Log any incidents on the 'Exam Room Record': computer changes, code re-issue, extended time, early departure, bathroom breaks — include time and details.

 Pre-registration required. No on-site sign-ups accepted.

Prohibited Items

- Printed references or dictionaries
- School bags or food
- Mobile phones (cannot be carried)
- Electronic media of any kind

Leaving the Seat

- Candidates must remain seated during exam
- Bathroom: must raise hand, get approval, register
- Only ONE candidate may be away at a time
- No discussing questions while away from seat

Scratch Paper

- Use blank space on 'Candidate Instructions' sheet
- Write name + PC number on every page
- Sheets collected upon exit — no extra paper provided

Desk Items Allowed

- Exam PC equipment only
- 'Candidate Instructions' sheet
- Writing instrument (pen/pencil)
- Valid photo ID document

5. Candidate Identity Verification

 **Begin verification 20 minutes after the exam starts (at 19:00)**

What to record	Enter candidate's Exam Access Code and Authentication Code into the attendance sheet. The Authentication Code is auto-generated on the candidate's screen after login.
University students	Verify with Student ID card.
Non-university candidates	Verify with photo-bearing National ID, Health Insurance Card, or Driver's License.
Disputed photo ID	Photograph the ID and candidate for evidence. Handle the issue after the exam concludes — do not interrupt the exam.
No ID present (unknown candidate)	Treat as disputed — photograph and document. Resolve post-exam.

6. Handling Technical Issues

PC Crash — Recoverable

- Restart the computer and relaunch CodingFrenzy
- If system restores OK: click 'Submit' button on the problem and enter proctor password to recover saved work (auto-saved every minute)

PC Crash — Unrecoverable

- Confirm a spare machine has network connectivity
- Move candidate to spare machine to continue
- If old Access Code fails, issue a new one (one code per candidate only)
- Allow exam to continue first — report the incident on JoinNet afterward

Extended Exam Time

- On the candidate machine: click 'Proctor Management' → enter password
- Under 'Candidate Machine Operations': adjust the time extension field (negative values allowed)
- Ensure the adjusted start/end window covers the current moment
- Report to main host via JoinNet: [venue], [candidate account], [minutes extended]

Score Sync Issue

- Candidate submission accepted on candidate machine, but proctor machine not updated
- Photograph the candidate machine screen and record the time
- After exam ends, email the evidence to the organizing university (NSYSU)

1

Candidate Exit Survey

URL appears when candidates exit the system. Ask them to complete it before leaving.

2

Verify Scores

Candidates must confirm their score on the scoreboard before leaving the exam room.

3

Collect Materials

Collect all 'Candidate Instructions' sheets (used as scratch paper) from every candidate.

4

Staff Survey

After the exam, all co-hosting school staff log in to the CPE website and complete the post-exam report questionnaire (even if no issues occurred).



Files to Upload to CPE Website



Attendance Sheets

Scan staff + candidate sheets into one multi-page PDF. Rename: [SchoolName].pdf



Submission Files

Compress C:\CodingFrenzy@CPEExam\Submits folder as ZIP/RAR. Rename: [SchoolName].zip



Exam Room Photos

1–3 photos of the exam room, compressed as ZIP/RAR. Rename: [SchoolName]photos.zip

8. Scoreboard & Key Resources

Live Scoreboard

- Candidates view scores on their exam machine
- Any non-exam computer can also view the scoreboard via browser
- URL format:
- <http://cpe0X.arping.me/cert/frenzy-records-exam.php>
- Replace 'cpe0X' with your school's assigned server (cpe01 – cpe06)
- Use the server indicated on your school's attendance sheet for load balancing

JoinNet Communication

- All venues communicate ONLY through JoinNet during the exam
- URL: <http://cu.nsysu.edu.tw/>
- Navigate to: 'University Programming Assessment' course → 'Course Discussion' → 'Live/Online Discussion Room'
- When ready: send '[School Name], exam room ready' via JoinNet
- NSYSU announces official start once ALL venues confirm ready
- Report any incidents (crashes, extensions, anomalies) via JoinNet immediately



Log into JoinNet and set up WebCam before 17:20



Log all incidents with time stamps on the Exam Room Record sheet



Candidates must sit in pre-assigned seats — no self-seating



In case of crash: use spare machine; report on JoinNet; extend time if needed



No phones or materials permitted inside the exam room



After exam: upload PDF, Submissions ZIP, and Photos ZIP to CPE website



Only ONE proctor machine per LAN — multiple causes screen flickering



Complete the post-exam staff survey on the CPE website



Verify IDs 20 min after exam start; record Access Code + Auth Code